

Build a Tiny Database in Java

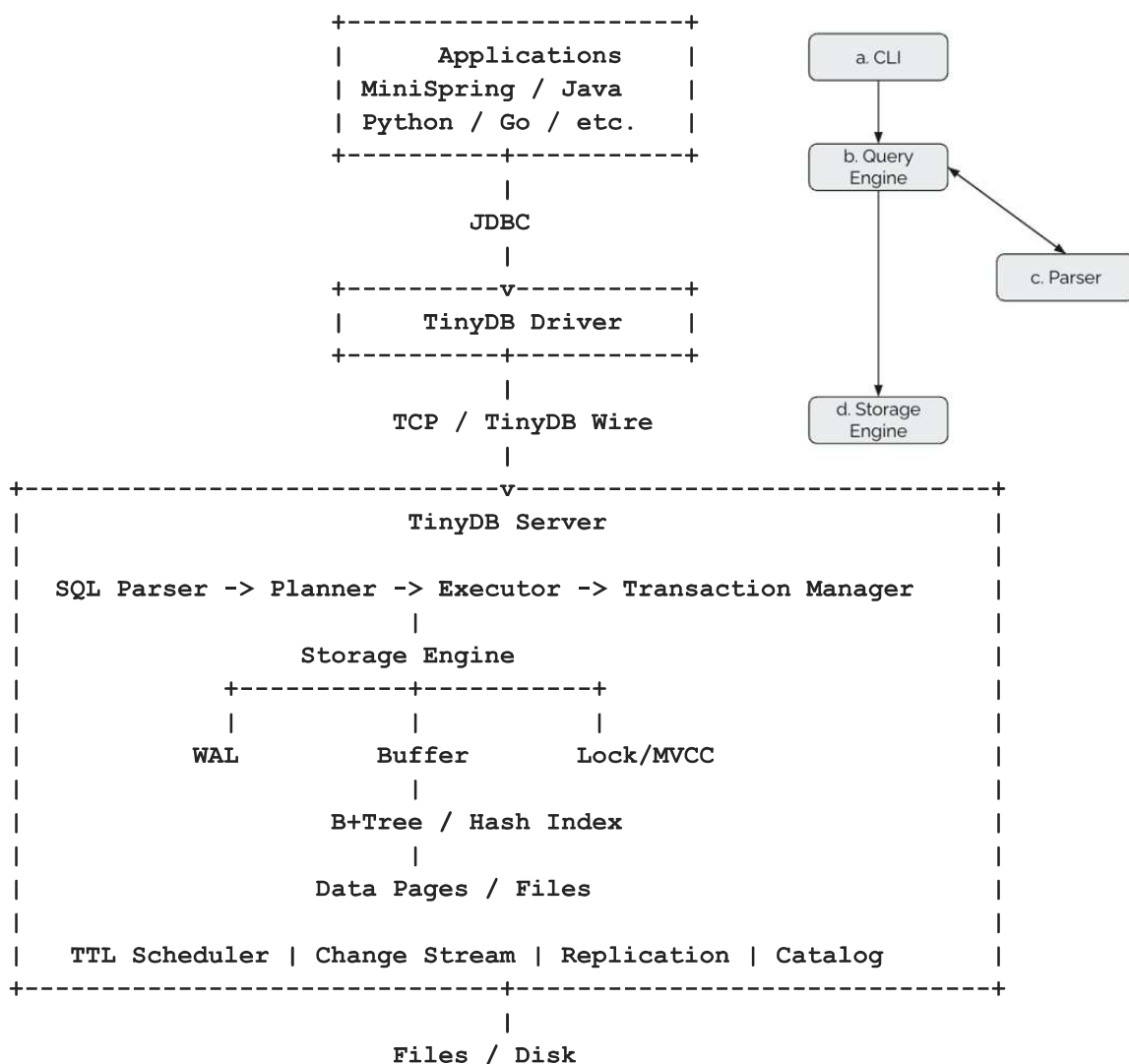
<https://medium.com/javarevisited/build-a-tiny-database-in-java-ca6d3f06e115>

```
Tinysql> SELECT A,B FROM T1 WHERE A=1;  
+---+-----+  
| a | b      |  
+---+-----+  
| 1 | Alice  |  
| 2 | Bob    |  
+---+-----+  
exit;
```

Components

1. CLI
2. Query Parser
3. Query Engine
4. Storage Engine

This code is a bare-minimum implementation designed for learning purposes. Basically, I would like information on the following:



1. Command Line Interface

In the **CLI loop**, we call the query engine `doQuery` or `doUpdate` function based on the SQL query's prefix keyword. Yes, it is pretty naive :).

*The query engine abstracts all the downstream interactions. **Query Engine** internally calls **Parser** to parse the SQL statements and convert it to Domain objects. The **Domain Objects** are then used by the **Query Planner** to create a **query execution plan** based on some rules or heuristics. The Query Plan is executed by the **Execution Engine**. The Execution engine talks to **Storage Engine** by **scanning** disk files and reading the table records. These records are then send back to Query Engine via **Iterator Pattern**. The Query Engine applies **filters** based on **Selection** and **Projection** Operators defined in the **Query Plan** and finally sends it back to the **CLI** for printing the table output.*

// TinyDbCli.java

```
private static void cliLoop(IQueryEngine qe) {
    Scanner scanner = new Scanner(System.in).useDelimiter(";");
    TablePrinter tablePrinter = new TablePrinter();
    while (true) {
        System.out.print("tinysql> ");
        String sql = scanner.next().replace("\n", " ").replace("\r", "").trim();

        TableDto result;
        if (sql.startsWith("exit")) break;
        else if (sql.startsWith("select")) result = qe.doQuery(sql);
        else result = qe.doUpdate(sql);

        if (result.message.isEmpty()) tablePrinter.print(result);
        else System.out.println(result.message);
    }
    scanner.close();
}
```

Phase 1 --- Build the In-Memory Database

Do not start with disk, networking, replication, and SQL simultaneously.

<pre>public record Column(String name, DataType type, boolean nullable) {} public record TableSchema(String tableName, List<Column> columns) {}</pre>	<pre>public enum DataType { INT, BIGINT, BOOLEAN, VARCHAR, DECIMAL, DATE, TIMESTAMP }</pre>
<pre>public final class Row { private final Map<String, Object> values; public Row(Map<String, Object> values) { this.values = Map.copyOf(values); } public Object get(String column) { return values.get(column); } public Set<String> columnNames() { return values.keySet(); } }</pre>	

Phase 2 --- Database Catalog

The catalog is the database's metadata system.

<pre>Database ├── DatabaseName ├── Tables ├── Indexes └── Metadata</pre>	<pre>public interface Catalog { void createDatabase(String name); void createTable(TableSchema schema); void dropTable(String tableName); TableSchema getTable(String tableName); boolean tableExists(String tableName); List<TableSchema> listTables(); }</pre>
--	---

Phase 3 --- Implement SQL Lexer

Before parsing SQL, tokenize it.

Input: SELECT id, name FROM employee WHERE salary > 50000;	SELECT IDENTIFIER(id) COMMA IDENTIFIER(name) FROM IDENTIFIER(employee)	WHERE IDENTIFIER(salary) GREATER_THAN NUMBER(50000) SEMICOLON
---	---	---

Create:

public enum TokenType { SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, FROM, WHERE, VALUES, INTO, SET, TABLE, DATABASE, }	AND, OR NOT, IDENTIFIER, STRING, NUMBER, EQ, NE, GT, GTE, LT, LTE, COMMA, DOT, LPAREN, RPAREN, STAR, EOF
---	--

Implement: Lexer, Token, TokenType

Phase 4 --- Build the SQL Parser

Build an Abstract Syntax Tree.

Define: public sealed interface SqlStatement permits SelectStatement, InsertStatement, UpdateStatement, DeleteStatement, CreateTableStatement, DropTableStatement { }	Example: // SQL: SELECT AGE,NAME FROM STUDENTS WHERE AGE=18; // AGE,NAME -> Fields/ columns // STUDENTS -> Table // AGE=18 -> Predicate public record SelectStatement (List<String> columns, String table, Expression where) implements SqlStatement {} // SQL: SELECT AGE,NAME FROM STUDENTS WHERE AGE=18; // tblName -> STUDENTS // flds -> NAME, AGE // vals -> "Bob", 18 // AGE=18 -> Predicate public record InsertStatement (List<String> columns, String table, List<D_Constant> vals) implements SqlStatement {}
--	---

SQL Grammar --- Start Small

Initial grammar:

<pre>statement := select insert update delete createTable dropTable select := SELECT selectList FROM identifier whereClause? selectList := STAR identifier (COMMA identifier)* whereClause := WHERE expression expression := comparison expression AND expression expression OR expression comparison := identifier operator value operator := = != > >= < <=</pre>	Execution Engine: <pre>SQL Lexer Parser AST Planner Execution Plan Executor Storage Engine</pre>
---	---

Then expand gradually.

Example:

<pre>public interface Executor { QueryResult execute(ExecutionPlan plan, TransactionContext transaction); }</pre>	Create executors: <pre>SelectExecutor InsertExecutor UpdateExecutor DeleteExecutor CreateTableExecutor DropTableExecutor</pre>
---	---

How the Lexer, Parser, and Executors Actually Connect

```
String sql = "INSERT INTO employee (id, name) VALUES (1, 'Alice')";
```

1. **Lexer** : sql -> List<Token>
2. **Parser** : List<Token> -> SqlStatement(a concrete InsertStatement)
3. **Engine** : SqlStatement -> pick the matching Executor
4. **Executor**: SqlStatement -> mutate/read StorageEngine -> QueryResult

The Lexer and Parser never know executors exist. The executors never know how the statement was parsed.

record InsertStatement (String table, List<String> columns, List<Object> values) implements SqlStatement {}	record UpdateStatement (String table, Map<String, Object> assignments, Expression where) implements SqlStatement {}	record DeleteStatement (String table, Expression where) implements SqlStatement {}
record CreateTableStatement (TableSchema schema) implements SqlStatement {}	record DropTableStatement (String table) implements SqlStatement {}	

```
sealed interface Expression
    permits Comparison, And, Or {}
```

```
record Comparison(String column, String operator, Object value) implements Expression {}
```

```
record And(Expression left, Expression right) implements Expression {}
```

```
record Or(Expression left, Expression right) implements Expression {}
```

```
public final class ExpressionEvaluator {
    public static boolean matches(Row row, Expression expression) {
        return switch (expression) {
            case Comparison c -> evaluateComparison(row, c);
            case And a -> matches(row, a.left()) && matches(row, a.right());
            case Or o -> matches(row, o.left()) || matches(row, o.right());
        };
    }

    private static boolean evaluateComparison(Row row, Comparison c) {
        Object actual = row.get(c.column());
        int cmp = compare(actual, c.value());
        return switch (c.operator()) {
            case "=" -> Objects.equals(actual, c.value());
            case "!=" -> !Objects.equals(actual, c.value());
            case ">" -> cmp > 0;
            case ">=" -> cmp >= 0;
            case "<" -> cmp < 0;
            case "<=" -> cmp <= 0;
            default -> throw new IllegalStateException("Unsupported operator: " + c.operator());
        };
    }
}
```

```

@SuppressWarnings("unchecked")
private static int compare(Object actual, Object expected) {
    if (actual instanceof Comparable<?> && expected != null) {
        return ((Comparable<Object>) actual).compareTo(expected);
    }
    return 0;
}
}

```

Finally, `QueryResult` and `TransactionContext`, both referenced by the `Executor` interface above but never defined until now:

```

public record QueryResult(List<Row> rows, int affectedRows) {
    public static QueryResult ofRows(List<Row> rows) {
        return new QueryResult(rows, rows.size());
    }

    public static QueryResult ofAffected(int count) {
        return new QueryResult(List.of(), count);
    }
}

```

```

public final class TransactionContext {

    // Deliberately empty for now. Real isolation, locking, and undo
    // arrive with the Transaction Manager (section 22). Until then,
    // every statement is its own implicit, auto-committed transaction.
    public static final TransactionContext AUTOCOMMIT = new TransactionContext();
}

```

A Minimal `StorageEngine` to Execute Against

```

public interface StorageEngine {
    void createTable(TableSchema schema);
    void dropTable(String tableName);
    void insert(String tableName, Row row);
    List<Row> scan(String tableName);
    void replaceRows(String tableName, List<Row> rows);
}

public final class MemoryStorageEngine implements StorageEngine {
    private final Map<String, List<Row>> tables = new ConcurrentHashMap<>();

    @Override
    public void createTable(TableSchema schema) {
        tables.putIfAbsent(schema.tableName(), new CopyOnWriteArrayList<>());
    }
    ....
}

```

The Six Executors

```
public interface StatementExecutor<S extends SqlStatement> {  
    QueryResult execute(S statement, TransactionContext transaction);  
}
```

SelectExecutor

```
public final class SelectExecutor implements StatementExecutor<SelectStatement> {  
    private final StorageEngine storageEngine;  
  
    public SelectExecutor(StorageEngine storageEngine) {  
        this.storageEngine = storageEngine;  
    }  
  
    @Override  
    public QueryResult execute(SelectStatement statement, TransactionContext transaction) {  
        List<Row> matching = storageEngine.scan(statement.table()).stream()  
            .filter(row -> statement.where() == null  
                || ExpressionEvaluator.matches(row, statement.where()))  
            .toList();  
  
        if (statement.columns().size() == 1 && statement.columns().get(0).equals("")) {  
            return QueryResult.ofRows(matching);  
        }  
  
        List<Row> projected = matching.stream()  
            .map(row -> projectColumns(row, statement.columns()))  
            .toList();  
        return QueryResult.ofRows(projected);  
    }  
  
    private Row projectColumns(Row row, List<String> columns) {  
        Map<String, Object> projected = new LinkedHashMap<>();  
        for (String column : columns) {  
            projected.put(column, row.get(column));  
        }  
        return new Row(projected);  
    }  
}
```

InsertExecutor

```
public final class InsertExecutor implements StatementExecutor<InsertStatement> {

    private final StorageEngine storageEngine;
    private final Catalog catalog;

    public InsertExecutor(StorageEngine storageEngine, Catalog catalog) {
        this.storageEngine = storageEngine;
        this.catalog = catalog;
    }

    @Override
    public QueryResult execute(InsertStatement statement, TransactionContext transaction) {
        TableSchema schema = catalog.getTable(statement.table());
        if (schema == null) {
            throw new IllegalStateException("No such table: " + statement.table());
        }
        if (statement.columns().size() != statement.values().size()) {
            throw new IllegalArgumentException("Column count does not match value count");
        }

        Map<String, Object> values = new LinkedHashMap<>();
        for (int i = 0; i < statement.columns().size(); i++) {
            values.put(statement.columns().get(i), statement.values().get(i));
        }
        validateAgainstSchema(schema, values);

        storageEngine.insert(statement.table(), new Row(values));
        return QueryResult.ofAffected(1);
    }

    private void validateAgainstSchema(TableSchema schema, Map<String, Object> values) {
        for (Column column : schema.columns()) {
            if (!column.nullable() && values.get(column.name()) == null) {
                throw new IllegalStateException(column.name() + " cannot be null");
            }
        }
    }
}
```

UpdateExecutor

```
public final class UpdateExecutor implements StatementExecutor<UpdateStatement> {
    private final StorageEngine storageEngine;

    public UpdateExecutor(StorageEngine storageEngine) { this.storageEngine = storageEngine; }

    @Override
    public QueryResult execute(UpdateStatement statement, TransactionContext transaction) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> updated = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            if (statement.where() == null || ExpressionEvaluator.matches(row, statement.where())) {
                Map<String, Object> newValues = new LinkedHashMap<>();
                for (String column : row.columnNames()) {
                    newValues.put(column, row.get(column));
                }
                newValues.putAll(statement.assignments());
                updated.add(new Row(newValues));
                affected++;
            } else {
                updated.add(row);
            }
        }

        storageEngine.replaceRows(statement.table(), updated);
        return QueryResult.ofAffected(affected);
    }
}
```

DeleteExecutor:

```
public final class DeleteExecutor implements StatementExecutor<DeleteStatement> {
    private final StorageEngine storageEngine;
    public DeleteExecutor(StorageEngine storageEngine) {this.storageEngine = storageEngine; }

    @Override
    public QueryResult execute(DeleteStatement statement, TransactionContext transaction) {
        List<Row> current = storageEngine.scan(statement.table());
        List<Row> kept = new ArrayList<>();
        int affected = 0;

        for (Row row : current) {
            boolean shouldDelete = statement.where() == null
                || ExpressionEvaluator.matches(row, statement.where());
            if (shouldDelete) {
                affected++;
            } else {
                kept.add(row);
            }
        }
        storageEngine.replaceRows(statement.table(), kept);
        return QueryResult.ofAffected(affected);
    }
}
```

CreateTableExecutor:

```
public final class CreateTableExecutor implements StatementExecutor<CreateTableStatement> {

    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public CreateTableExecutor(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public QueryResult execute(CreateTableStatement statement, TransactionContext transaction) {
        if (catalog.tableExists(statement.schema().tableName())) {
            throw new IllegalStateException("Table already exists: " + statement.schema().tableName());
        }
        catalog.createTable(statement.schema());
        storageEngine.createTable(statement.schema());
        return QueryResult.ofAffected(0);
    }
}
```

DropTableExecutor:

```
public final class DropTableExecutor implements StatementExecutor<DropTableStatement> {

    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public DropTableExecutor(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public QueryResult execute(DropTableStatement statement, TransactionContext transaction) {
        if (!catalog.tableExists(statement.table())) {
            throw new IllegalStateException("No such table: " + statement.table());
        }
        catalog.dropTable(statement.table());
        storageEngine.dropTable(statement.table());
        return QueryResult.ofAffected(0);
    }
}
```

Wiring It Together: The QueryEngine Dispatcher

```
public final class QueryEngine {

    private final SelectExecutor selectExecutor;
    private final InsertExecutor insertExecutor;
    private final UpdateExecutor updateExecutor;
    private final DeleteExecutor deleteExecutor;
    private final CreateTableExecutor createTableExecutor;
    private final DropTableExecutor dropTableExecutor;

    public QueryEngine(Catalog catalog, StorageEngine storageEngine) {
        this.selectExecutor = new SelectExecutor(storageEngine);
        this.insertExecutor = new InsertExecutor(storageEngine, catalog);
        this.updateExecutor = new UpdateExecutor(storageEngine);
        this.deleteExecutor = new DeleteExecutor(storageEngine);
        this.createTableExecutor = new CreateTableExecutor(catalog, storageEngine);
        this.dropTableExecutor = new DropTableExecutor(catalog, storageEngine);
    }

    public QueryResult run(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize(); // section 6
        SqlStatement statement = new SqlParser(tokens).parseStatement(); // section 7
        return execute(statement, TransactionContext.AUTOCOMMIT);
    }

    public QueryResult execute(SqlStatement statement, TransactionContext transaction) {
        return switch (statement) {
            case SelectStatement s -> selectExecutor.execute(s, transaction);
            case InsertStatement s -> insertExecutor.execute(s, transaction);
            case UpdateStatement s -> updateExecutor.execute(s, transaction);
            case DeleteStatement s -> deleteExecutor.execute(s, transaction);
            case CreateTableStatement s -> createTableExecutor.execute(s, transaction);
            case DropTableStatement s -> dropTableExecutor.execute(s, transaction);
        };
    }
}
```

Alternative: ANTLR-Generated Parser + the Visitor Pattern

```
// Uses an ANTLR-generated MySQLLexer/MySQLStatementParser instead of a
// hand-written one. ANTLR turns a .g4 grammar file into the lexer,
public class MySqlParser implements IParser {
    MySqlStatementVisitor sqlStatementVisitor;

    public MySqlParser(String sql) {
        MySQLLexer lexer = new MySQLLexer(CharStreams.fromString(sql));
        var parser = new MySQLStatementParser(new CommonTokenStream(lexer));

        sqlStatementVisitor = new MySqlStatementVisitor(parser);
        sqlStatementVisitor.visit(parser.execute());
    }

    @Override
    public QueryData queryCmd() {return (QueryData) sqlStatementVisitor.getValue();}

    @Override
    public Object updateCmd() {return sqlStatementVisitor.getValue(); }
}

// The Visitor walks ANTLR's generated parse tree and extracts exactly
// the fields needed into a domain object -- the ANTLR equivalent of
// this guide's SqlStatement records.
public class MySqlStatementVisitor extends MySQLStatementBaseVisitor {
    private final MySQLStatementParser parser;
    private COMMAND_TYPE commandType;
    private String tableName;

    @Override
    public Object visitCreateTable(MySQLStatementParser.CreateTableContext ctx) {
        commandType = COMMAND_TYPE.CREATE_TABLE;
        return super.visitCreateTable(ctx);
    }

    @Override
    public Object visitTableName(MySQLStatementParser.TableNameContext ctx) {
        this.tableName = ctx.name().getText();
        return super.visitTableName(ctx);
    }
}
```

The pipeline is structurally identical to section 10.1's, just built from generated pieces instead of hand-written ones:

This guide (hand-written):	ANTLR-based (e.g. ShardingSphere):
SQL text	SQL text
SqlLexer (hand-written)	MySQLLexer (generated from a grammar)
SqlParser (hand-written)	MySQLStatementParser (generated) + a
	Visitor subclass that walks its parse tree
SqlStatement (a sealed	QueryData / InsertData / ModifyData
interface of records)	(plain domain objects the Visitor builds)
QueryEngine.execute(...)	dispatch by commandType / object type
(an exhaustive switch)	(structurally the same idea, without
	a sealed-type-checked switch

Phase 5 --- Storage Engine

The first version can use:
Map<tableName, List<Row>>

java.nio.file.Path
FileChannel
ByteBuffer

Store data as pages.

Recommended initial page size: **4096 bytes** or, **8192 bytes**

Page-Based Storage

```
public final class Page {  
    public static final int PAGE_SIZE = 8192;  
  
    private final long pageld;  
    private final ByteBuffer buffer;  
}
```

A page should eventually contain:

Page Header

pageld
pageType
LSN
freeSpaceOffset
recordCount
checksum

Records

Record 1
Record 2
Record 3
...

Record Format

Record

recordLength
transactionId
flags
columnCount

column1
column2
column3
...

Row Identifier:

```
public record RowId(  
    long pageld,  
    int slotId  
) {}
```

Then:

RowId -> Page -> Slot -> Record

Indexes can point to RowId instead of copying complete rows.

Buffer Pool

Do not read the same page from disk repeatedly.

```
public interface BufferPool {  
    Page get(Pageld pageld);  
    void markDirty(Pageld pageld);  
    void flush(Pageld pageld);  
    void flushAll();  
}
```

Use an eviction strategy.
Initially: **LRU**

Later:

Clock
2Q
LFU
adaptive policies

Free Space Management

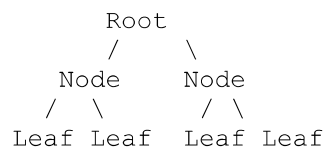
The database must know where a new record can be inserted.

Implement: **FreePageList** or **Free Space Map**

Example:
Page 10 -> 30% free
Page 11 -> 5% free
Page 12 -> 70% free

An insert can select Page 12.

B+Tree



Leaf nodes contain: **key -> RowId**

Example:

100 -> (page=20, slot=3)
101 -> (page=20, slot=4)
102 -> (page=21, slot=1)

The Three Parts of a Query Engine

Query Optimizer	-- decides HOW to answer a query (which plan, which indexes)
Execution Engine	-- actually reads rows/columns according to that plan
Catalog / Metadata	-- knows what tables and indexes exist, and their shape Manager

The Plan Interface: Relational Algebra Plus Cost

```
public interface Plan {  
    Scan open();  
    int blocksAccessed();  
    int recordsOutput();  
    TableSchema schema();  
}
```

TablePlan

```
public final class TablePlan implements Plan {

    private final String tableName;
    private final StorageEngine storageEngine;
    private final Catalog catalog;

    public TablePlan(String tableName, StorageEngine storageEngine, Catalog catalog) {
        this.tableName = tableName;
        this.storageEngine = storageEngine;
        this.catalog = catalog;
    }

    @Override
    public Scan open() {
        return storageEngine.openScan(tableName); // section 20.2 -- the ONE seam into physical
storage
    }

    @Override
    public int blocksAccessed() {
        return catalog.getStatistics(tableName).blockCount(); // fed by whatever tracks physical page
counts, sections 12-16
    }

    @Override
    public int recordsOutput() {
        return catalog.getStatistics(tableName).recordCount();
    }

    @Override
    public TableSchema schema() {
        return catalog.getTable(tableName);
    }
}
```

Similarly, we will have **SelectPlan** and **ProjectPlan**

BasicQueryPlanner and UpdatePlanner: From SqlStatement to a Plan

```
public interface QueryPlanner {

    Plan createPlan(SelectStatement statement, TransactionContext tx);
}
```

```

public final class BasicQueryPlanner implements QueryPlanner {

    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public BasicQueryPlanner(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public Plan createPlan(SelectStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog); // Step 1: read

        if (statement.where() != null) {
            plan = new SelectPlan(plan, statement.where()); // Step 2: filter rows
        }
        boolean selectStar = statement.columns().size() == 1 && statement.columns().get(0).equals("");
        if (!selectStar) {
            plan = new ProjectPlan(plan, statement.columns()); // Step 3: filter columns
        }
        return plan;
    }
}

```

```

public interface UpdatePlanner {

    int executeInsert(InsertStatement statement, TransactionContext tx);
    int executeUpdate(UpdateStatement statement, TransactionContext tx);
    int executeDelete(DeleteStatement statement, TransactionContext tx);
    int executeCreateTable(CreateTableStatement statement, TransactionContext tx);
    int executeDropTable(DropTableStatement statement, TransactionContext tx);
}

```

```

public final class BasicUpdatePlanner implements UpdatePlanner {
    private final Catalog catalog;
    private final StorageEngine storageEngine;

    public BasicUpdatePlanner(Catalog catalog, StorageEngine storageEngine) {
        this.catalog = catalog;
        this.storageEngine = storageEngine;
    }

    @Override
    public int executeInsert(InsertStatement statement, TransactionContext tx) {
        Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
        Scan scan = plan.open(); // section 20 -- a WRITABLE scan
        scan.insert(); // position at a fresh slot
        for (int i = 0; i < statement.columns().size(); i++) {
            scan.setVal(statement.columns().get(i), statement.values().get(i));
        }
        scan.close();
        return 1;
    }
}

```

```

@Override
public int executeUpdate(UpdateStatement statement, TransactionContext tx) {
    Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
    if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
    Scan scan = plan.open();
    int affected = 0;
    while (scan.next()) {
        for (var entry : statement.assignments().entrySet()) {
            scan.setVal(entry.getKey(), entry.getValue());
        }
        affected++;
    }
    scan.close();
    return affected;
}

@Override
public int executeDelete(DeleteStatement statement, TransactionContext tx) {
    Plan plan = new TablePlan(statement.table(), storageEngine, catalog);
    if (statement.where() != null) plan = new SelectPlan(plan, statement.where());
    Scan scan = plan.open();
    int affected = 0;
    while (scan.next()) {
        scan.delete();
        affected++;
    }
    scan.close();
    return affected;
}

@Override
public int executeCreateTable(CreateTableStatement statement, TransactionContext tx) {
    catalog.createTable(statement.schema());
    storageEngine.createTable(statement.schema());
    return 0;
}

@Override
public int executeDropTable(DropTableStatement statement, TransactionContext tx) {
    catalog.dropTable(statement.table());
    storageEngine.dropTable(statement.table());
    return 0;
}
}

```

Notice **executeUpdate/executeDelete** reuse **SelectPlan** for their WHERE clause -- filtering which rows to mutate is the identical operation as filtering which rows to read, so it is composed from the same **Plan**, not reimplemented.

BasicPlanner and QueryEngine: Wiring Parsing, Planning, and Transactions

```
public final class BasicPlanner {

    private final QueryPlanner queryPlanner;
    private final UpdatePlanner updatePlanner;

    public BasicPlanner(QueryPlanner queryPlanner, UpdatePlanner updatePlanner) {
        this.queryPlanner = queryPlanner;
        this.updatePlanner = updatePlanner;
    }

    public Plan createQueryPlan(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        if (!(statement instanceof SelectStatement select)) {
            throw new IllegalArgumentException("Not a query: " + sql);
        }
        return queryPlanner.createPlan(select, tx);
    }

    public int executeUpdate(String sql, TransactionContext tx) {
        SqlStatement statement = parse(sql);
        return switch (statement) {
            case InsertStatement s -> updatePlanner.executeInsert(s, tx);
            case UpdateStatement s -> updatePlanner.executeUpdate(s, tx);
            case DeleteStatement s -> updatePlanner.executeDelete(s, tx);
            case CreateTableStatement s -> updatePlanner.executeCreateTable(s, tx);
            case DropTableStatement s -> updatePlanner.executeDropTable(s, tx);
            case SelectStatement s -> throw new IllegalArgumentException("Use createQueryPlan for a
SELECT");
        };
    }

    private SqlStatement parse(String sql) {
        List<Token> tokens = new SqlLexer(sql).tokenize(); // section 6
        return new SqlParser(tokens).parseStatement(); // section 7
    }
}
```

This is the exact same exhaustive-**switch**-over-a-sealed-interface trick section 10.4 already used for **QueryEngine.execute** -- adding a **CREATE INDEX** statement later means the compiler refuses to build this method until a matching case is added here too, same as before.

BasicQueryEngine

The externally-visible database facade -- what a CLI (section 41-42) or a JDBC driver (section 37) actually calls:

```
public final class BasicQueryEngine {

    private final BasicPlanner planner;
    private final TransactionManager transactionManager; // section 22

    public BasicQueryEngine(BasicPlanner planner, TransactionManager transactionManager) {
        this.planner = planner;
        this.transactionManager = transactionManager;
    }

    public QueryResult doQuery(String sql) {
        TransactionContext tx = transactionManager.begin();
        Plan plan = planner.createQueryPlan(sql, tx);
        Scan scan = plan.open();

        List<Row> rows = new ArrayList<>();
        while (scan.next()) {
            Map<String, Object> values = new LinkedHashMap<>();
            for (String field : plan.schema().fieldNames()) {
                values.put(field, scan.getVal(field));
            }
            rows.add(new Row(values));
        }
        scan.close();
        transactionManager.commit(tx);
        return QueryResult.ofRows(rows);
    }

    public QueryResult doUpdate(String sql) {
        TransactionContext tx = transactionManager.begin();
        int affected = planner.executeUpdate(sql, tx);
        transactionManager.commit(tx);
        return QueryResult.ofAffected(affected);
    }
}
```

doQuery is the only place left that walks a **Scan** into a **List<Row>** -- and it does that only because the CLI/JDBC layer above it wants a materialized result set to print or return. The engine itself, all the way down through **Plan/Scan**, never materializes more than one row at a time (section 20.1 explains exactly why that matters).

The Catalog Manager: Persisting Schema as Data

tinydb_tables			tinydb_columns			
tblname	slotsize		tblname	fldname	type	length
employee	48		employee	id	BIGINT	0
			employee	name	VARCHAR	50

```

public final class PersistentCatalog implements Catalog {
    private static final TableSchema TABLES_SCHEMA = new TableSchema(
        "tinydb_tables", List.of(new Column("tblname", VARCHAR, false), new Column("slotsize", INT, false))
    );

    private static final TableSchema COLUMNS_SCHEMA = new TableSchema(
        "tinydb_columns", List.of(
            new Column("tblname", VARCHAR, false), new Column("fldname", VARCHAR, false),
            new Column("type", DataType.VARCHAR, false), new Column("length", DataType.INT, false)
        )
    );

    private final StorageEngine storageEngine;

    public PersistentCatalog(StorageEngine storageEngine) {
        this.storageEngine = storageEngine;
        storageEngine.createTable(TABLES_SCHEMA); // bootstrap: the catalog's own tables must exist first
        storageEngine.createTable(COLUMNS_SCHEMA);
    }

    @Override
    public void createTable(TableSchema schema) {
        storageEngine.insert(TABLES_SCHEMA.tableName(),
            new Row(Map.of("tblname", schema.tableName(), "slotsize", estimateSlotSize(schema))));
        for (Column column : schema.columns()) {
            storageEngine.insert(COLUMNS_SCHEMA.tableName(), new Row(Map.of(
                "tblname", schema.tableName(),
                "fldname", column.name(),
                "type", column.type().name(),
                "length", 0)));
        }
    }

    @Override
    public TableSchema getTable(String tableName) {
        boolean exists = storageEngine.scan(TABLES_SCHEMA.tableName()).stream()
            .anyMatch(row -> row.get("tblname").equals(tableName));
        if (!exists) return null;

        List<Column> columns = storageEngine.scan(COLUMNS_SCHEMA.tableName()).stream()
            .filter(row -> row.get("tblname").equals(tableName))
            .map(row -> new Column((String) row.get("fldname"),
                DataType.valueOf((String) row.get("type")), true))
            .toList();
        return new TableSchema(tableName, columns);
    }
}

```

TablePhysicalLayout: From Logical Schema to Byte Offsets

```
public final class TablePhysicalLayout {

    private final TableSchema schema;
    private final Map<String, Integer> offsets;
    private final int slotSize;

    public TablePhysicalLayout(TableSchema schema) {
        this.schema = schema;
        this.offsets = new HashMap<>();
        int position = 0;
        for (Column column : schema.columns()) {
            offsets.put(column.name(), position);
            position += sizeInBytes(column);
        }
        this.slotSize = position;
    }

    private int sizeInBytes(Column column) {
        return switch (column.type()) {
            case INT -> 4;
            case BIGINT -> 8;
            case BOOLEAN -> 1;
            case VARCHAR -> 50; // fixed-length slots for simplicity, even VARCHAR gets a fixed budget
            case DECIMAL -> 12;
            case DATE, TIMESTAMP -> 8;
        };
    }

    public int offset(String fieldName) { return offsets.get(fieldName); }
    public int slotSize() { return slotSize; }
}
```

Query Operators

Model execution as operators:	Example:	Plan:
TableScan IndexScan Filter Project Sort Limit Aggregate HashJoin NestedLoopJoin	SELECT name FROM employee WHERE salary > 50000 ORDER BY salary DESC LIMIT 10	Limit Sort Filter salary > 50000 TableScan employee

The Scan Interface: Streaming Rows Instead of Materializing Lists

```
public interface Scan {
    boolean next();
    Object getVal(String fieldName);
    boolean hasField(String fieldName);
    void close();
    // Writable operations -- see the note below on Interface Segregation
    void insert();
    void setVal(String fieldName, Object value);
    void delete();
}
```

TableScan

TableScan is the only Scan that talks to StorageEngine directly -- everything else wraps another Scan:

```
// MemoryStorageEngine.openScan -- still backed by the in-memory List<Row> for now.
// Once page-based storage (sections 11-16) exists, this walks Pages via RowId (section 14)
// instead of a Java Iterator -- the Scan CONTRACT above does not change either way.
@Override
public Scan openScan(String tableName) {
    List<Row> rows = tables.get(tableName);
    return new Scan() {
        private int index = -1;

        @Override public boolean next() { index++; return index < rows.size(); }

        @Override public Object getVal(String fieldName) { return rows.get(index).get(fieldName); }

        @Override
        public boolean hasField(String fieldName) {
            return rows.get(index).columnNames().contains(fieldName);
        }

        @Override
        public void insert() { rows.add(new Row(new HashMap<>())); index = rows.size() - 1;}

        @Override
        public void setVal(String fieldName, Object value) {
            Map<String, Object> updated = new LinkedHashMap<>();
            for (String column : rows.get(index).columnNames())
                updated.put(column, rows.get(index).get(column));
            updated.put(fieldName, value);
            rows.set(index, new Row(updated));
        }

        @Override
        public void delete() {
            rows.remove(index);
            index--; // the next next() call must not skip the row that just slid into this slot
        }

        @Override
        public void close() { }
    };
}
```

```
}
```

SelectScan

TableScan is the only Scan that talks to StorageEngine directly -- everything else wraps another Scan:

```
public final class SelectScan implements Scan {

    private final Scan input;
    private final Expression predicate; // section 10.1

    public SelectScan(Scan input, Expression predicate) {
        this.input = input;
        this.predicate = predicate;
    }

    @Override
    public boolean next() {
        while (input.next()) {
            if (matchesCurrentRow()) return true;
        }
        return false;
    }

    private boolean matchesCurrentRow() { // see the ExpressionEvaluator extension just below
        return ExpressionEvaluator.matchesScan(this, predicate);
    }

    @Override public Object getVal(String fieldName) { return input.getVal(fieldName); }
    @Override public boolean hasField(String fieldName) { return input.hasField(fieldName); }
    @Override public void setVal(String fieldName, Object value) { input.setVal(fieldName, value); }
    @Override public void insert() { input.insert(); }
    @Override public void delete() { input.delete(); }
    @Override public void close() { input.close(); }
}
```

SelectScan needs to evaluate the same **Expression** tree section 10.1 already built -- but against a Scan's current position, not a fully-formed Row. Extend ExpressionEvaluator with a second entry point that mirrors **matches(Row, Expression)** field for field, reading through **Scan.getVal(...)** instead of **Row.get(...)**:

```
public static boolean matchesScan(Scan scan, Expression expression) {
    return switch (expression) {
        case Comparison c -> evaluateComparison(scan.getVal(c.column()), c);
        case And a -> matchesScan(scan, a.left()) && matchesScan(scan, a.right());
        case Or o -> matchesScan(scan, o.left()) || matchesScan(scan, o.right());
    };
}

private static boolean evaluateComparison(Object actual, Comparison c) {
    int cmp = compare(actual, c.value());
    return switch (c.operator()) {
        case "=" -> Objects.equals(actual, c.value());
        case "!=" -> !Objects.equals(actual, c.value());
        case ">" -> cmp > 0;
        case ">=" -> cmp >= 0;
        case "<" -> cmp < 0;
        case "<=" -> cmp <= 0;
    };
}
```

```

        default -> throw new IllegalStateException("Unsupported operator: " + c.operator());
    };
}

```

ProjectScan

wraps a child Scan and a field list -- it filters columns, and explicitly rejects access to any column outside the projection, exactly the same "not part of this schema" guard `ProjectPlan.schema()` (section 19.2) already narrows to:

```

public final class ProjectScan implements Scan {

    private final Scan input;
    private final List<String> fields;

    public ProjectScan(Scan input, List<String> fields) {
        this.input = input;
        this.fields = fields;
    }

    @Override
    public boolean next() { return input.next(); }

    @Override
    public Object getVal(String fieldName) {
        if (!hasField(fieldName)) throw new RuntimeException("field " + fieldName + " not found.");
        return input.getVal(fieldName);
    }

    @Override
    public boolean hasField(String fieldName) { return fields.contains(fieldName); }

    @Override
    public void setVal(String fieldName, Object value) {
        throw new UnsupportedOperationException("read-only projection");
    }
    @Override public void insert() { throw new UnsupportedOperationException("read-only projection"); }
    @Override public void delete() { throw new UnsupportedOperationException("read-only projection"); }
    @Override public void close() { input.close(); }
}

```

ProjectScan throwing on every write method is worth noticing -- projection is fundamentally a read-shaping operation, and section 20.1's "split into `ReadOnlyScan`/`UpdatableScan`" refinement would let this class simply not implement the write methods at all, rather than implementing them just to reject every call.

Transaction Manager

```

public interface TransactionManager {
    Transaction begin();
    void commit(Transaction transaction);
    void rollback(Transaction transaction);
}

```

ACID

```

Atomicity    -> WAL + transaction state + undo/MVCC
Consistency -> constraints + transaction rules
Isolation    -> locks and/or MVCC

```

Durability -> WAL + fsync + recovery

Write-Ahead Logging (WAL)

WAL is one of the most important components.

Rule:

The database must persist the log record before the corresponding dirty data page is considered durable.

Example:
UPDATE employee
SET salary = 120000
WHERE id = 1;

WAL:

BEGIN TX 10

UPDATE
table=employee
row=1
oldSalary=100000
newSalary=120000

COMMIT TX 10

Disk ordering:
1. Write WAL
2. Flush WAL
3. Write data page
4. Flush data page later

If the server crashes after step 2:
- WAL exists
- data page may not
Recovery can reconstruct the intended state.

Log Sequence Number (LSN)

Give every WAL record an LSN.

```
public record Lsn(long value) {}
```

Every page stores its latest applied LSN.

During recovery:

if WAL LSN > page LSN
replay WAL

WAL Record Types

Start with:
BEGIN
INSERT
UPDATE
DELETE
COMMIT
ABORT
CHECKPOINT

Later:
CREATE_TABLE
DROP_TABLE
CREATE_INDEX
DROP_INDEX
ALTER_TABLE

Rollback Strategy

Approach A --- Undo logging

Store: old value

For: UPDATE employee SET salary = 120000 WHERE id = 1;	Record: old = 100000 new = 120000
---	---

Rollback writes: **salary = 100000**

Approach B --- MVCC

Keep multiple row versions.

employee id=1 Version 1: salary=100000 xmin=10 xmax=20 Version 2: salary=120000 xmin=20 xmax=null	Transactions see the version appropriate to their snapshot. Recommendation: implement a simple undo/WAL system first, then evolve toward MVCC.
---	--

MVCC

MVCC means Multi-Version Concurrency Control.

Instead of modifying one row in place: old row new row version	Track: xmin xmax transaction status	Example: Row Version A xmin = 10 xmax = 20 Row Version B xmin = 20 xmax = null
---	--	--

Transaction 10 can see A.

Transaction 20 can see B after the appropriate visibility rules.

Isolation Levels

Support:	Each isolation level requires actual visibility/locking behavior.
READ UNCOMMITTED READ COMMITTED REPEATABLE READ SERIALIZABLE	READ COMMITTED -> statement-level snapshot REPEATABLE READ -> transaction-level snapshot SERIALIZABLE -> strict locking or serializable MVCC

Lock Manager

<pre>public interface LockManager { void acquire(TransactionId transactionId, ResourceId resource, LockMode mode); void releaseAll(TransactionId transactionId); }</pre>	Lock modes: SHARED EXCLUSIVE
--	--

Compatibility: S + S -> allowed S + X -> blocked X + S -> blocked X + X -> blocked	Later implement: row locks page locks table locks lock escalation deadlock detection deadlock timeout
--	---

Deadlock Detection

TX1 -> TX2 TX2 -> TX1 Cycle means deadlock.	Create a: Wait-for Graph Detect cycles periodically. Abort one transaction.
--	---

Checkpoints

Without checkpoints, recovery may have to replay a huge WAL.

Create: **CHECKPOINT**

Procedure:

1. Flush dirty pages
2. Flush WAL
3. Write checkpoint record
4. Persist checkpoint metadata

Crash Recovery

On startup:

```
TinyDB
|
RecoveryManager
|
Read checkpoint
|
Read WAL
|
Redo committed operations
|
Undo incomplete operations
|
Rebuild temporary state
|
Open database
```

Test it by deliberately killing the process:

```
insert
update
kill -9
restart
```

Durability Tests

Create tests for:

```
commit + crash
rollback + crash
partial WAL write
partial page write
multiple transactions
crash during checkpoint
crash during index update
```

Build a JDBC Driver

Implement the JDBC interfaces needed by your supported feature set.

Start with:	<pre>public final class TinyDbDriver implements java.sql.Driver { } JDBC URL: jdbc:tinydb://localhost:9090/company Connection connection = DriverManager.getConnection("jdbc:tinydb://localhost:9090/company");</pre>
Driver Connection Statement PreparedStatement ResultSet DatabaseMetaData ResultSetMetaData SQLException	

Why Not HTTP?

You can expose an HTTP API: **POST /query**
but HTTP should not be the core database protocol.
For a database client/server architecture use:

TCP + framed protocol

TinyDB Terminal

Add commands: .help .tables .databases .schema employee .describe employee .indexes .transactions .version .exit	history arrow-key navigation tab completion command history formatted result sets	Interactive SQL Shell The CLI should support multiline SQL: <pre>tinydb> SELECT * -> FROM employee -> WHERE id = 10; Detect: ;</pre>
--	---	---

Native DBeaver Protocol Support

```
DBeaver
|
PostgreSQL/MySQL driver
|
TinyDB PostgreSQL/MySQL-compatible protocol
|
TinyDB server
```

This is a separate project phase. Do this only after:

```
SQL engine
storage
transactions
JDBC
CLI
```

TLS

Eventually support:

<pre>TinyDB client TLS TinyDB server</pre>	Configuration: <pre>tinydb.tls.enabled=true tinydb.tls.keystore=...</pre>
--	--

TTL

TTL is useful for:

<pre>sessions temporary records caches event retention OTP records temporary application data</pre>	For automatic expiration, create: TTLManager Store expiration metadata in the row: expiresAt
---	---

TTL Index

A better implementation uses a time-ordered structure:

```
expiration timestamp -> RowId
```

For example: 12:00 -> row A 12:01 -> row B 12:04 -> row C	The TTL worker scans only expired entries. Do not scan the entire database every second.
---	--

Change Data Capture / Streams

Now implement DynamoDB-style change streams. Every committed mutation generates an event:

INSERT
UPDATE
DELETE

Example:

```
{
  "eventId": "evt-100",
  "transactionId": "tx-20",
  "table": "employee",
  "operation": "UPDATE",
  "key": {
    "id": 1
  },
  "oldImage": {
    "salary": 100000
  },
  "newImage": {
    "salary": 120000
  },
  "timestamp": "..."
}
```

Stream Architecture

```
SQL
|
Transaction
|
WAL
|
Commit
|
ChangeEvent
|
StreamLog
|
+-----+-----+-----+
|         |         |         |
Consumer  Consumer  Consumer
```

Important: **Publish database change events only after the transaction reaches the required commit point.**

Stream Log

Implement an append-only log:

```
stream-000001.log
stream-000002.log
...
```

Each event gets: **sequenceNumber**

Consumers maintain:

consumer -> last processed sequence

Example:

```
consumerA -> 100
consumerB -> 95
consumerC -> 70
```

Stream APIs

SQL:

```
CREATE STREAM employee_changes
ON employee;
```

```
ChangeStream stream =
    tinyDb.stream("employee_changes");
```

```
stream.subscribe(event -> {
    System.out.println(event);
});
```

Stream Retention

Configure:

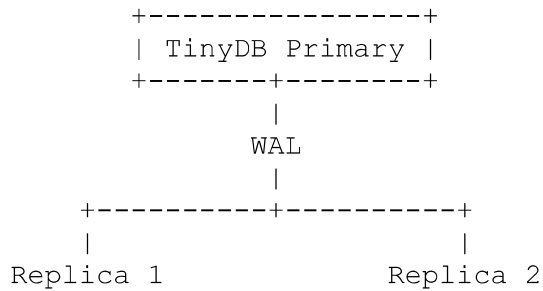
```
tinydb.stream.retention=24h  
or:  
tinydb.stream.retention.bytes=10GB
```

Deletion policy:

keep events until:
time retention OR
size retention

Replication

Replication should not be added before WAL.



- The primary produces WAL.
- Replicas consume WAL records.

Replication Types

Stage1: Async primary -> replica.

```
Primary
|
| WAL
v
Replica
```

Stage2: Replica acknowledgements.

```
Primary
|
+---- Replica A
|
+---- Replica B
```

Stage3: Synchronous replication.

Commit may require:
local WAL flush
+
replica acknowledgement

Stage4: Leader election / failover.

This is significantly harder.

Threading Model

Network threads
Query execution threads
WAL writer
Checkpoint thread
TTL worker
Replication worker
Stream worker
Background maintenance

Backpressure

Streams and replication can overwhelm consumers.

Implement:
bounded queues
rate limits
batching
timeouts
consumer lag monitoring